利用 MCP 进行 EDA 仿真的 Root Cause 分析

原创 芯球上面 芯球上面 2026年2月28日 07:07 美国

> **作者按**：芯片验证工程师平均将 40% 的时间花在 debug 上——不是在思考，而是在 log、波形、RTL 之间来回搬运信息。这篇文章介绍如何用 MCP 协议让 Claude 接管这部分工作，自主完成从仿真报错到定位根因的全流程。代码已开源，拿来即用。

开篇之前感谢一下@寻迹 提供的灵感。这次代码工程是笔者跟claude聊了将近一整天才实现，期间推翻了三次MCP架构，经历四次上下文压缩，不然单次对话信息量太大claude记不住所有内容。我们开始吧。

一、开篇：一个熟悉的场景

仿真跑完，打开 log。

几十行 assertion fail，时间戳密密麻麻。你知道接下来要做什么——启动波形工具，等它加载那个 1GB 的波形文件，把信号一个个拖进波形窗口，时间轴拖到第一个报错时刻，再切回来看 RTL，再切回去对波形……

如果今天只有一个 case 失败，这套流程勉强能接受。

但现实往往是：8 个 case 有 ERROR，每个背后的根因不同。今天是解密路径出问题，明天是状态机跳转不对，后天是总线协议违例。问题不重复，但定位流程高度雷同——

看 log → 开波形工具 → 找信号 → 翻波形 → 对照 RTL → 推理根因。

这个流程里，真正需要你动脑的只有最后一步。

前面那些——提取报错时刻、搜索信号路径、查特定时刻的信号值——本质上是体力活。工具已经给了你答案，你只是在做信息搬运。

这篇文章想聊的，是怎么把这部分体力活交出去，让 AI 接手从「看 log」到「定位嫌疑代码行」的全过程，你只需要在最后做判断。

> 📸 **【图1】** Claude Code 中 MCP 工具调用链展开的界面，显示 parse_sim_log → search_signals → analyze_failures 的完整调用过程

二、什么是 MCP

在写这个工具之前，先解释一个概念。

Claude 默认能做什么？读文件、写代码、回答问题。但它拿不到实时数据，访问不了你的仿真环境，更不知道你的 FSDB 里某个信号在某个时刻是什么值。

MCP（Model Context Protocol） 解决的就是这个问题。它是 Anthropic 推出的一个开放协议，允许你给 Claude 注册外部工具——Claude 在推理过程中如果需要某个数据，会主动调用对应的工具，拿到结果后继续分析。

打个比方：没有 MCP 的 Claude 像一个只能在房间里思考的顾问，你把资料带进来它才能看。有了 MCP，它可以主动打电话、查数据库、调接口——**工具由你定义，调用由它决定。**

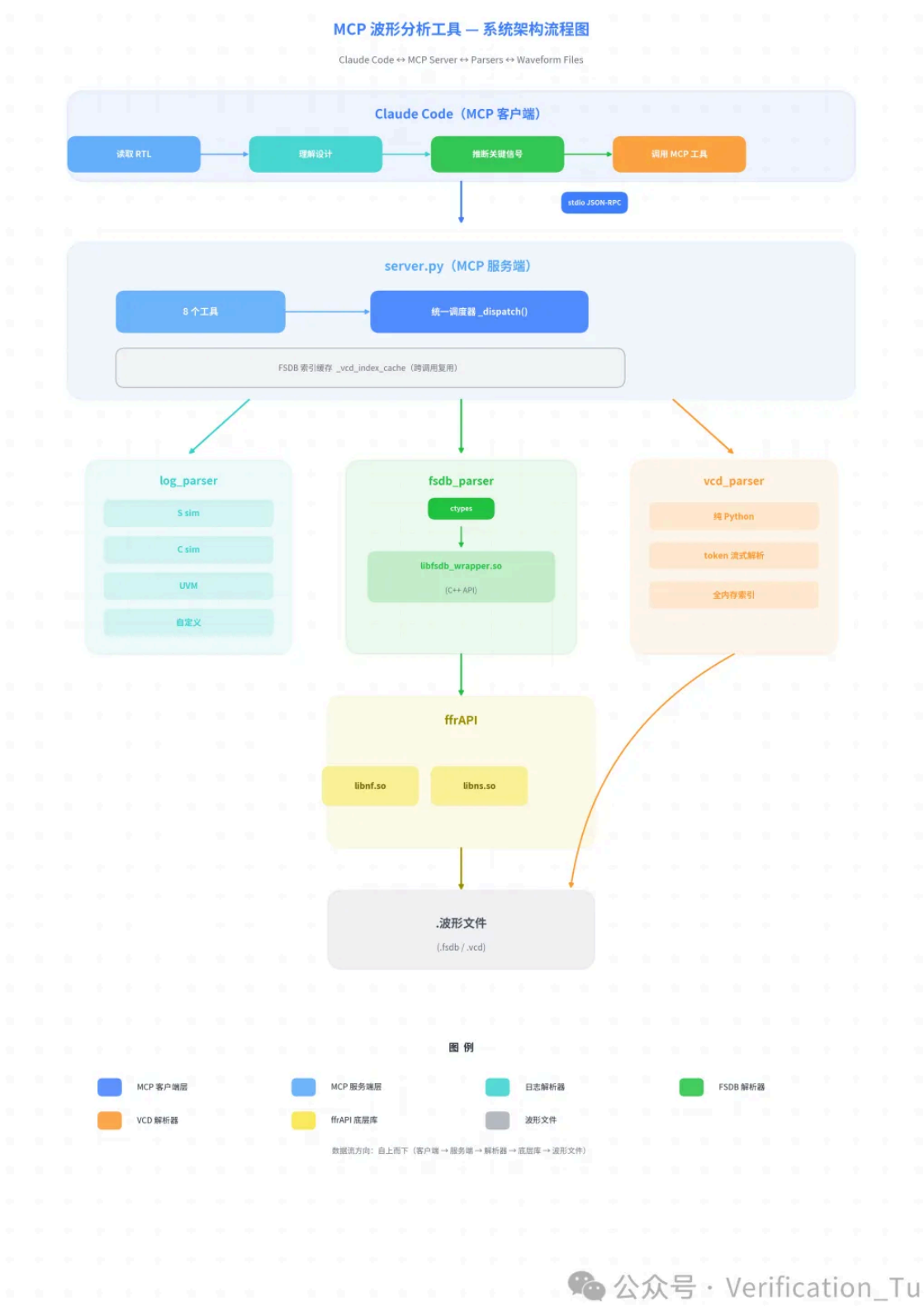
对 EDA 场景来说，这意味着什么？

你可以把「解析仿真 log」、「搜索信号路径」、「查询波形数据」这些操作封装成 MCP 工具。Claude 在分析问题的过程中，会自己决定什么时候调用哪个工具、用什么参数——就像一个会用波形调试工具的工程师，而不是一个只能看你贴过来截图的顾问。

就像我在前面文章所说，Claude Code以CLI方式运行，并不能自行打开波形进行debug，我们可以通过截图的方式把GUI发给Claude，但这种方式需要耗费大量token，我在高考志愿那篇的尝试全程用Claude Desktop实现，期间出现了2次token耗尽的情况。而直接分析波形文件就像@寻迹说的，占用大量的上下文，而MCP正是 Claude 推出的用于解决此类问题的利器。

> Claude并不能自行打开波形进行debug
>
> 芯球上面，公众号：芯球上面
> AI芯片设计师来了！Claude Code完成SPI设计验证闭环实战

MCP server 本身没有什么神秘的地方，就是一个普通的 Python 程序。你定义工具，它负责跟 Claude 通信，剩下的交给 Claude 来驱动。



公众号 · Verification_Tu

> 📸 **【图2】**：MCP 架构示意图

三、Waveform MCP 的设计思路

明确了 MCP 是什么之后，下一个问题是：针对 EDA 调试场景，这个工具应该做哪些事？

把验证工程师定位 root cause 的过程拆开来看，本质上是三类操作：

- 📋 **读 log**：从几千行仿真输出里提取有效报错，拿到时间、文件、行号
- 📊 **读波形**：在特定时刻查某个信号的值，或者看它在一段时间内怎么跳变
- 🔍 **关联分析**：把报错时刻和波形数据对齐，结合 RTL 逻辑推断根因

Waveform MCP 把这三类操作封装成 **8 个工具**，覆盖从「拿到路径」到「联合分析」的全流程：

```
get_sim_paths          → 根据项目结构返回 log 和波形路径
parse_sim_log          → 解析 log，提取结构化报错列表
search_signals         → 在波形中按关键字搜索信号完整路径
get_signal_at_time     → 查询某信号在某时刻的值
get_signal_transitions → 获取信号完整跳变历史
get_signals_around_time → 多信号在某时刻前后的快照
get_waveform_summary   → 波形文件基本信息
analyze_failures       → 核心：log + 波形联合分析
```

其中 `analyze_failures` 是整个工具链的核心。它对 log 里的每一条报错，自动提取对应时刻的波形上下文，把两份数据打包交给 Claude。

关键设计：为什么返回两份波形数据？

这里有一个设计决策值得单独说。每条报错的波形上下文返回**两份数据**，而不是一份：

数据	内容	用途
wave_context	报错时刻前后几纳秒的信号快照	快速定位异常现象
signal_history_before_fail	从仿真开始到报错时刻的完整跳变	追溯根因

为什么要两份？**因为很多 bug 的根因不在报错时刻，而在更早。** 状态机在 T1 时刻走错了路，assertion 在 T2 时刻才失败——如果只看 T2 前后的窗口，你永远找不到 T1 发生了什么。

完整历史让 Claude 具备真正的因果推理能力，而不只是描述报错时刻的现象。

整体数据流

```
仿真 log +  波形文件
         |
         ▼
parse_sim_log    → 提取所有报错（时间、文件、行号、消息）
search_signals   → 确认关键信号的完整层级路径
analyze_failures → 对每条报错，自动对齐波形数据
         |
         ▼
Claude 结合 RTL 代码交叉推理
         |
         ▼
Root Cause 结论：是 RTL bug 还是 TB 问题，定位到具体代码行
```

```
.
├── build_wrapper.sh
├── config.py
├── custom_patterns.yaml
├── fsdb_wrapper.cpp
├── libfsdb_wrapper.so
├── __pycache__
├── pytest.ini
├── README.md
├── server.py
├── src
└── tests

3 directories, 8 files
```

> 📸 **【图3】** 项目目录结构截图

四、实战：Bug 的定位过程

首先是一个断言类型报错自主定位，你会发现笔误在claude面前真的无处遁形

- 📋 **读 log**

报了很多这种类型的断言错误

```
"/home/robin/Projects/mcp_demo/tb/../tb/sva_top.sv", 66: top_tb.sva_top_inst.apUNEXPECTED_ASSERTION: started at 270000ps failed at 290000ps
    Offending '(s_bits == STATE1)'
"/home/robin/Projects/mcp_demo/tb/../tb/sva_top.sv", 79: top_tb.sva_top_inst.apEXPECTED_ASSERTION_1: started at 270000ps failed at 290000ps
    Offending '(s_bits == STATE1)'
"/home/robin/Projects/mcp_demo/tb/../tb/sva_top.sv", 66: top_tb.sva_top_inst.apUNEXPECTED_ASSERTION: started at 290000ps failed at 310000ps
    Offending '(s_bits == STATE1)'
"/home/robin/Projects/mcp_demo/tb/../tb/sva_top.sv", 66: top_tb.sva_top_inst.apUNEXPECTED_ASSERTION: started at 370000ps failed at 390000ps
    Offending '(s_bits == STATE1)'
"/home/robin/Projects/mcp_demo/tb/../tb/sva_top.sv", 66: top_tb.sva_top_inst.apUNEXPECTED_ASSERTION: started at 390000ps failed at 410000ps
    Offending '(s_bits == STATE1)'
"/home/robin/Projects/mcp_demo/tb/../tb/sva_top.sv", 66: top_tb.sva_top_inst.apUNEXPECTED_ASSERTION: started at 410000ps failed at 430000ps
    Offending '(s_bits == STATE1)'
"/home/robin/Projects/mcp_demo/tb/../tb/sva_top.sv", 66: top_tb.sva_top_inst.apUNEXPECTED_ASSERTION: started at 430000ps failed at 450000ps
    Offending '(s_bits == STATE1)'
"/home/robin/Projects/mcp_demo/tb/../tb/sva_top.sv", 66: top_tb.sva_top_inst.apUNEXPECTED_ASSERTION: started at 550000ps failed at 570000ps
    Offending '(s_bits == STATE1)'
```

- 🔍 **关联分析**

关联的rtl以及断言代码如下，这个问题其实挺简单，大家发现了吗

```
    always@(*)
        case(s_bits)
            IDLE: begin
                if(start) next_state = STATE1;
            end
            STATE1: begin
                next_state = STATE2;
            end
            STATE2: begin
                if(!start) next_state = IDLE;
            end
        endcase

    property pUNEXPECTED_ASSERTION;
            @(posedge clk)
            start |=> s_bits == STATE1 ##1 s_bits == STATE2;
    endproperty: pUNEXPECTED_ASSERTION
    apUNEXPECTED_ASSERTION: assert property (pUNEXPECTED_ASSERTION);

    property pEXPECTED_ASSERTION;
            @(posedge clk)
            (start && s_bits==IDLE) |=> s_bits == STATE1 ##1 s_bits == STATE2;
    endproperty: pEXPECTED_ASSERTION
    apEXPECTED_ASSERTION: assert property (pEXPECTED_ASSERTION);


    property pEXPECTED_ASSERTION_1;
            @(posedge clk)
            $rose(start) |=> s_bits == STATE1 ##1 s_bits == STATE2;
    endproperty: pEXPECTED_ASSERTION_1
    apEXPECTED_ASSERTION_1: assert property (pEXPECTED_ASSERTION_1);

```

claude的分析如下

● 波形数据确认了我的分析。下面是完整的 Root Cause 分析报告。

Root Cause 分析报告

仿真共报出 **59 个错误**，分 3 类，涉及 2 个独立的问题域。

错误 1: apUNEXPECTED_ASSERTION — 34 次 (sva_top.sv:66)

Assertion 代码：
```
property pUNEXPECTED_ASSERTION;
    @(posedge clk)
    start |=> s_bits == STATE1 ##1 s_bits == STATE2;
endproperty
```

Root Cause：Assertion 前置条件不充分（缺少状态守卫）

该 assertion 的含义是：**任何时刻** start==1，下一拍 s_bits 必须是 STATE1，再下一拍必须是 STATE2。

但 FSM 的状态转移逻辑是：
- IDLE + start → STATE1
- STATE1 → STATE2
- STATE2 + !start → IDLE

当 start==1 但 FSM 处于 **STATE1 或 STATE2** 时，s_bits 不会变成 STATE1，assertion 自然失败。

波形验证（第一次失败 @270ns→290ns）：

时刻	start	s_bits
250ns	1	STATE2(100)
270ns	1	STATE2(100) ← assertion 在此触发

公众号 · Verification_Tu

总结

错误类型	次数	问题所在	根因
apUNEXPECTED_ASSERTION	34	sva_top.sv:66	Assertion 前置条件缺少 s_bits==IDLE 守卫
apEXPECTED_ASSERTION_1	10	sva_top.sv:79	$rose(start) 仍可在非 IDLE 状态触发
UVM_ERROR [TOP]	15	top_tb.sv:125	两个独立随机进程的值天然不同步

注意：apEXPECTED_ASSERTION（line 72）使用了 start && s_bits==IDLE 作为前置条件，**0 次失败**，验证了前两个 assertion 的修复方向是正确的。

另外，sva_top.sv 的 FSM 组合逻辑（line 49-60）中存在 **latch 隐患**：IDLE + !start 和 STATE2 + start 分支未赋值 next_state，会推断出 latch。建议补全 default 分支或补全所有条件的赋值。

公众号 · Verification_Tu

这是典型的断言先行算子出错，另外claude还报了rtl组合逻辑存在latch隐患的问题

大家可能觉得这个太简单了，而且断言名字都已经暴露，claude可能通过读断言名字就已经定位出问题了。

我们来看一个真实的例子。

测试对象是一个 **DES 加解密 IP**，330 个标准测试向量，先跑加密，再跑解密。仿真结束后打开 log：

```
Running Encrypt test ...    ← 全部通过，0 个 ERROR

Running Decrypt test ...    ← 部分失败
ERROR: @2389500000 Expected 2e8653104f3834ea Got 2fd4571d1b3f34a3
ERROR: @2404300000 Expected 20b9e767b2fb1456 Got 70d8b2e7a2b9403a
...（共 15 个 ERROR）
```

加密全过，解密部分失败。如果是你，下一步会怎么做？

Step 1：提取报错，发现规律

Claude 调用 `parse_sim_log`，把报错列表结构化提取出来。拿到数据后，它注意到：失败的 15 个向量**分散**在整个解密测试集里，不是连续的，也不是集中在开头或结尾。

这个分布给了一个重要提示：**问题不是解密路径整体坏掉了，而是某个条件触发的偶发性错误。**

Step 2：读 RTL，理解设计逻辑

Claude 读取 `des.v`，理解 DES Feistel 网络的 16 轮迭代结构：

```
IP（初始置换）→ 16 轮 L/R 交换 → FP（终结置换）

每轮核心运算：Rout = Xin ^ out
```

Step 3：定位异常代码行

读到 `des.v` 第 298 行时，Claude 停了下来：

```
// des.v:298  ← 发现异常
assign Rout = Xin ^ out ^ {31'b0, (roundSel == 4'hd) & decrypt & (L[1:4]
//                          ↑ 这个额外的 XOR 项不应该存在
```

标准 DES 轮函数只有 `Xin ^ out` ，这里**多了一个条件 XOR 项。**

Step 4：分析触发条件，验证 Bug 机制

Claude 分析这个额外项的三个触发条件：

条件	含义
roundSel == 4'hd	第 13 轮迭代
decrypt == 1	解密模式
L[1:4] == 4'hA	L 寄存器高 4 位为 0xA

三个条件同时满足时， `Rout` 最低位被强制翻转，导致解密结果出错。

这完美解释了所有现象：

- 加密时 `decrypt=0` ，条件永假，所以**加密全过**
- 解密时只有中间状态恰好满足 `L[1:4]==0xA` 的向量才出错，所以是**部分失败**

Root Cause 确认，修复只需一行

```
// ❌ BUG: 多余的条件 XOR 项
assign Rout = Xin ^ out ^ {31'b0, (roundSel == 4'hd) & decrypt & (L[1:4]

// ✅ 修复: 删除多余条件
assign Rout = Xin ^ out;
```

从读 log 到定位到具体代码行，Claude **全程自主完成**，没有人工介入。

> 📷 **【图4】**： Claude Code 中完整的分析过程，展示从调用 parse_sim_log 到最终给出 des.v:298 结论的推理链

可以关注一下图5最后那个时间，从下任务给claude，到root cause，总用时1m23s

Root Cause 分析

错误现象

从 log 中可以看到：
- **Encrypt test 全部通过**（无任何 ERROR）
- **Decrypt test 有 15 个 ERROR**，期望值和实际输出不匹配

Root Cause: des.v 第 298 行存在恶意/错误的 XOR 注入

```
// des.v:298 — BUG 在这里
assign Rout = Xin ^ out ^ {31'b0, (roundSel == 4'hd) & decrypt & (L[1:4] == 4'hA)};
```

标准 DES 的 Feistel 轮运算中，Rout 应该只是 Xin ^ out。但这里多了一个额外的 XOR 项：

```
{31'b0, (roundSel == 4'hd) & decrypt & (L[1:4] == 4'hA)}
```

这个额外项会在**同时满足以下三个条件**时翻转 Rout 的最低位 (LSB)：

条件	含义
roundSel == 4'hd	第 13 轮
decrypt == 1	解密模式
L[1:4] == 4'hA	L 寄存器高 4 位等于 0xA

为什么 Encrypt 通过而 Decrypt 失败

- 加密时 decrypt == 0，该项恒为 0，不影响结果 → Encrypt 全部 PASS
- 解密时 decrypt == 1，当 round 13 时 L 的高 4 位恰好为 0xA，就会翻转 LSB 导致最终输出错误 → 部分 Decrypt 向量 FAIL（取决于中间状态是否触发条件）

修复方法

将 des.v:298 改为标准的 Feistel 运算：

```
assign Rout = Xin ^ out;
```

即删除 ^ {31'b0, (roundSel == 4'hd) & decrypt & (L[1:4] == 4'hA)} 这个多余的 XOR 项。

需要我帮你修复这个 bug 吗？

公众号 · Verification_Tu

✻ Sautéed for 1m 23s

📷 **【图5】**：Claude 给出的 root cause 结论文字，清晰标注 bug 位置和触发条件

五、与传统方式的对比

环节	传统方式（人工 + 波形工具）	MCP 方式（AI + 工具链）
读 log	手动打开，眼找 ERROR	parse_sim_log 自动提取结构化报错
看波形	启动波形工具，手动加信号，拖时间轴	程序化查询，指定信号和时刻直接返回值
关联分析	脑中比对 log 时间和波形跳变	analyze_failures 自动对齐报错与波形
读 RTL	看代码理解逻辑	Claude 同时读代码和波形，交叉推理
迭代	改代码 → 重跑 → 再开 波形工具	改代码 → 重跑 → 自动分析，闭环

需要说明的是，MCP 方式和波形工具 **并不是替代关系**。Claude 适合做第一遍的快速缩小范围——从几十个报错里找到最可疑的信号和代码区域。真正需要深入看某段波形细节、做跨层级信号追踪时，Verdi 依然是更合适的工具。

两者的定位是：**AI 负责信息汇总和初步推理，工程师负责最终判断和深度验证。**

六、结语

这套工具的完整代码已开源，包含 MCP server、C++ wrapper、log 解析器和单元测试，按 README 配置好 工具路径即可使用。

验证工程师的核心价值从来不是在 log 里找 ERROR，也不是把时间轴拖到正确的位置。这些事情，可以让工具去做。

📎 **GitHub 链接**：https://github.com/gokeshenzhen/waveform_mcp

如果这篇文章对你有帮助，欢迎转发给同事。有问题或建议，欢迎在评论区交流。

声明：本文仅供技术学习交流参考，不涉及任何授权工具使用方式。推荐使用正版或开源方案。

芯球上面

喜欢作者

修改于2026年3月1日